

Threat Modeling Report

Created on 4/26/2026 8:23:23 PM

Threat Model Name:

Owner:

Reviewer:

Contributors:

Description:

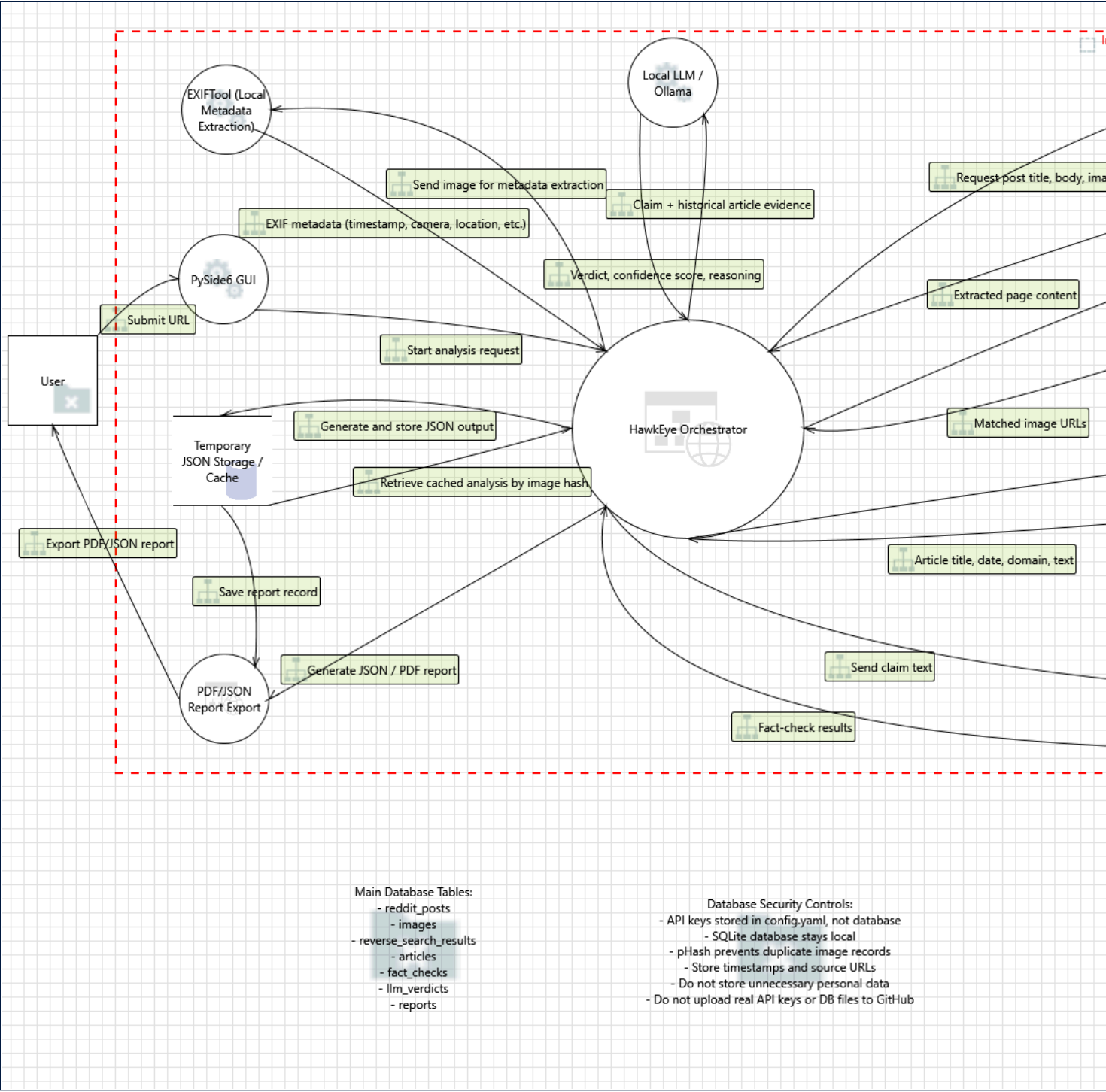
Assumptions:

External Dependencies:

Threat Model Summary:

Not Started	98
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	98
Total Migrated	0

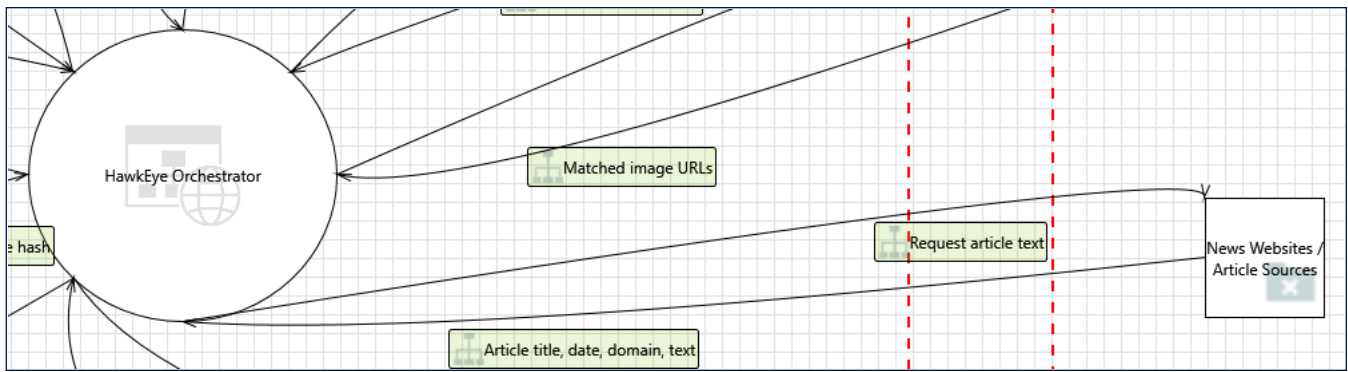
Diagram: HawkEye Database Architecture



HawkEye Database Architecture Diagram Summary:

Not Started	98
Not Applicable	0
Needs Investigation	0
Mitigation Implemented	0
Total	98
Total Migrated	0

Interaction: Article title, date, domain, text



1. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

2. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

3. An adversary may spoof News Websites / Article Sources and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

4. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

5. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdat#autocomplete-input

Mitigation(s): Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

6. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

Mitigation(s): https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

7. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing

Mitigation(s): Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

8. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

9. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data

Mitigation(s): Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

10. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

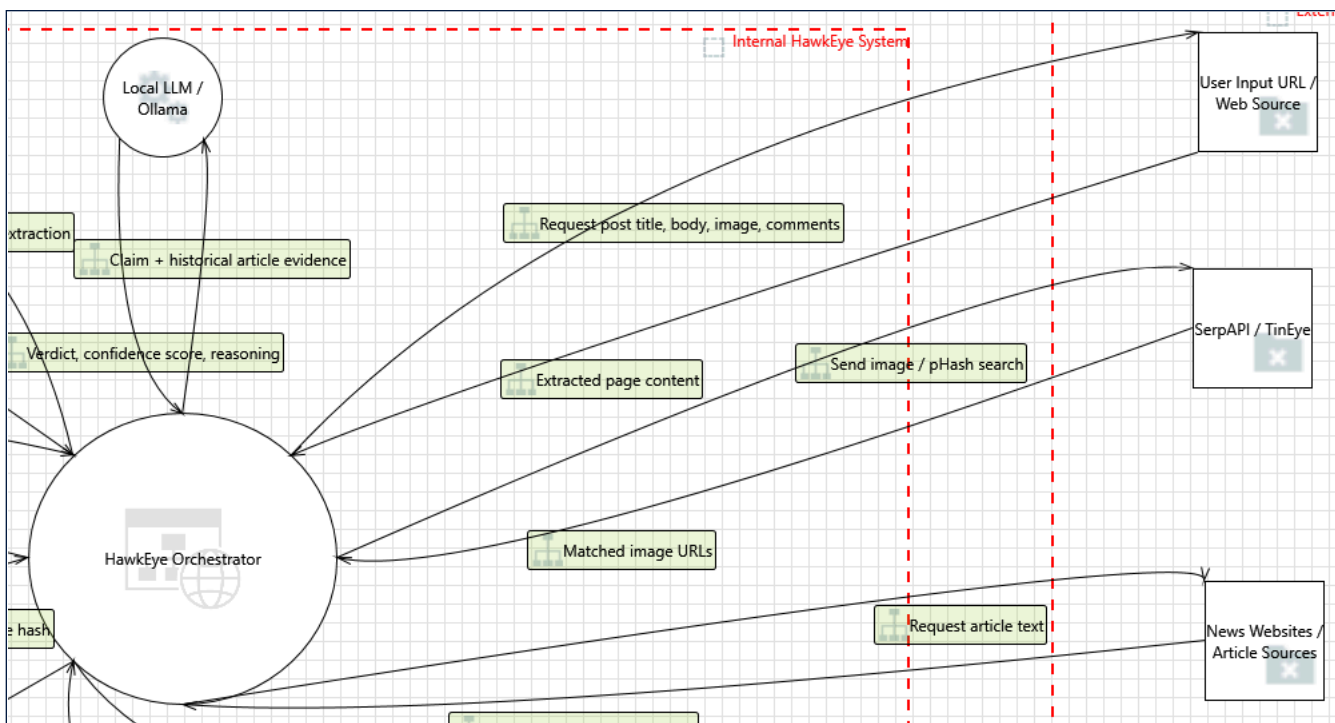
Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

Interaction: Extracted page content



11. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

Mitigation(s): https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

12. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

Mitigation(s): https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

13. An adversary may spoof User Input URL / Web Source and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

14. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

15. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

16. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

17. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

18. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

19. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data

Mitigation(s): Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

20. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

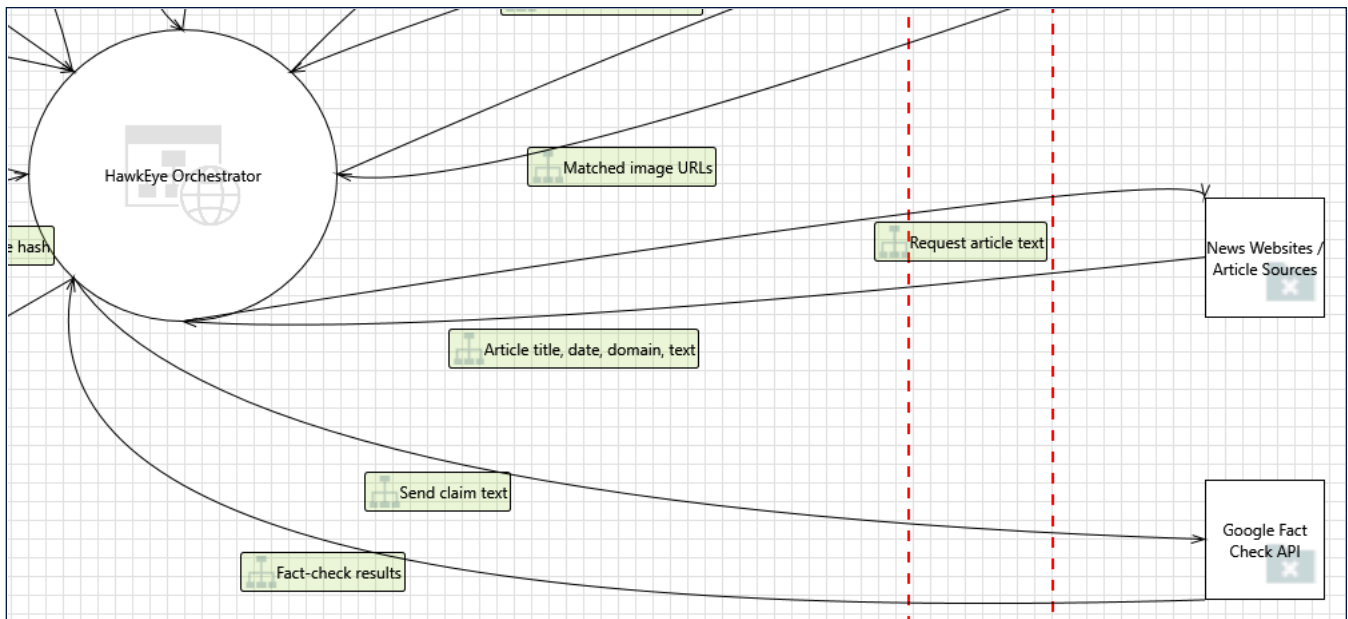
Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

Interaction: Fact-check results



21. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data
SDL Phase: Implementation

22. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering
Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe
SDL Phase: Implementation

23. An adversary may spoof Google Fact Check API and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing
Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application
Justification: <no mitigation provided>
Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app
SDL Phase: Design

24. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing
Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASPNET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

25. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdat#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtdat#input-validation Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtdat#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtdat#step-up-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtdat#forgot-pwd-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtdat#pwd-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtdat#string-method

SDL Phase: Implementation

26. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-ssl

SDL Phase: Implementation

27. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

28. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASPNET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

29. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data

Mitigation(s): Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

30. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

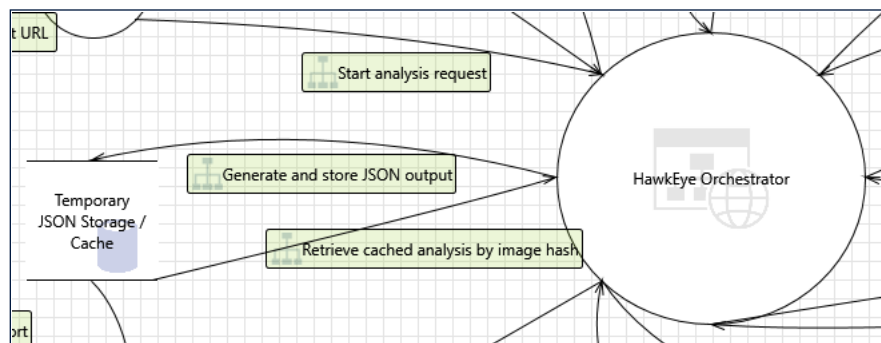
Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

Interaction: Generate and store JSON output



31. An adversary may leverage the lack of monitoring systems and trigger anomalous traffic to database [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary may leverage the lack of intrusion detection and prevention of anomalous database activities and trigger anomalous traffic to database

Justification: <no mitigation provided>

Possible Enable Threat detection on Azure SQL database. Refer: https://aka.ms/tmtauditlog#threat-detection

Mitigation(s):

SDL Phase: Design

32. An adversary can tamper critical database securables and deny the action [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can tamper critical database securables and deny the action

Justification: <no mitigation provided>

Possible Add digital signature to critical database securables. Refer: https://aka.ms/tmtcrypto#securables-db

Mitigation(s):

SDL Phase: Design

33. An adversary can deny actions on database due to lack of auditing [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that login auditing is enabled on SQL Server. Refer: https://aka.ms/tmtauditlog#identify-sensitive-entities

SDL Phase: Implementation

34. An adversary can gain access to sensitive data by performing SQL injection [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that login auditing is enabled on SQL Server. Refer: https://aka.ms/tmtauditlog#identify-sensitive-entities Ensure that least-privileged accounts are used to connect to Database server. Refer: https://aka.ms/tmtauthz#privileged-server Enable Threat detection on Azure SQL database. Refer: https://aka.ms/tmtauditlog#threat-detection Do not use dynamic queries in stored procedures. Refer: https://aka.ms/tmtinputval#stored-proc

SDL Phase: Implementation

35. An adversary can gain access to sensitive PII or HBI data in database [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: Additional controls like Transparent Data Encryption, Column Level Encryption, EKM etc. provide additional protection mechanism to high value PII or HBI data.

Justification: <no mitigation provided>

Possible Mitigation(s): Use strong encryption algorithms to encrypt data in the database. Refer: https://aka.ms/tmtcrypto#strong-db Ensure that sensitive data in database columns is encrypted. Refer: https://aka.ms/tmtdata#db-encrypted Ensure that database-level encryption (TDE) is enabled. Refer: https://aka.ms/tmtdata#tde-enabled Ensure that database backups are encrypted. Refer: https://aka.ms/tmtdata#backup Use SQL server EKM to protect encryption keys. Refer: https://aka.ms/tmtcrypto#ekm-keys Use AlwaysEncrypted feature if encryption keys should not be revealed to Database engine. Refer: https://aka.ms/tmtcrypto#keys-engine

SDL Phase: Implementation

36. An adversary can gain access to sensitive data by sniffing traffic to database [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can eaves drop on communication between application server and Temporary JSON Storage / Cache server, due to clear text communication protocol usage.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure SQL server connection encryption and certificate validation. Refer: https://aka.ms/tmtcommsec#sqlserver-validation Force Encrypted communication to SQL server. Refer: https://aka.ms/tmtcommsec#encrypted-sqlserver

SDL Phase: Implementation

37. An adversary can gain unauthorized access to database due to loose authorization rules [State: Not Started] [Priority: High]

Category: Elevation of Privileges

Description: Database access should be configured with roles and privilege based on least privilege and need to know principle.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that least-privileged accounts are used to connect to Database server. Refer: https://aka.ms/tmtauthz#privileged-server Implement Row Level Security RLS to prevent tenants from accessing each others data. Refer: https://aka.ms/tmtauthz#rls-tenants Sysadmin role should only have valid necessary users. Refer: https://aka.ms/tmtauthz#sysadmin-users

SDL Phase: Implementation

38. An adversary can gain unauthorized access to database due to lack of network access protection [State: Not Started] [Priority: High]

Category: Elevation of Privileges

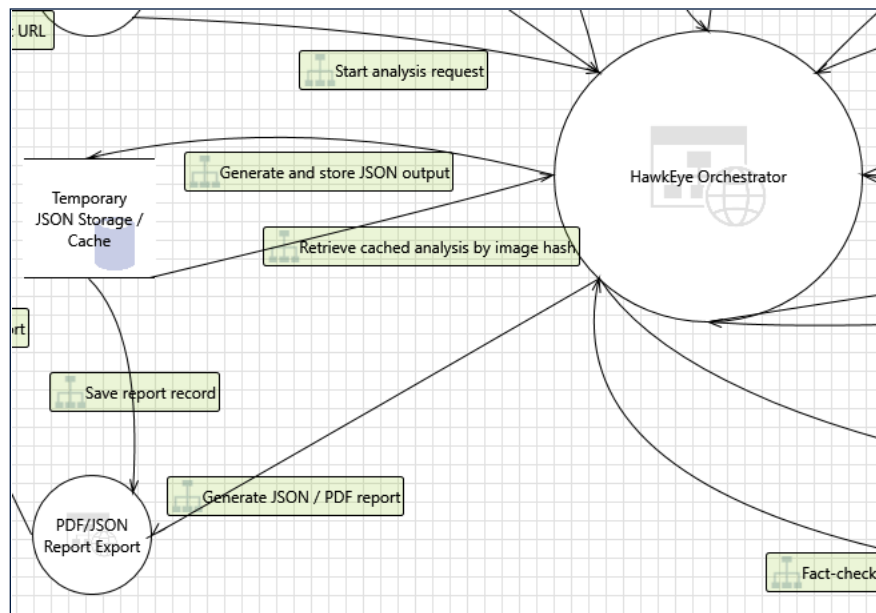
Description: If there is no restriction at network or host firewall level, to access the database then anyone can attempt to connect to the database from an unauthorized location

Justification: <no mitigation provided>

Possible Mitigation(s): Configure a Windows Firewall for Database Engine Access. Refer: https://aka.ms/tmtconfigmgmt#firewall-db

SDL Phase: Implementation

Interaction: Generate JSON / PDF report



39. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

40. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

41. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

42. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

43. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

44. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

45. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

46. An adversary may spoof HawkEye Orchestrator and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

47. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

48. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

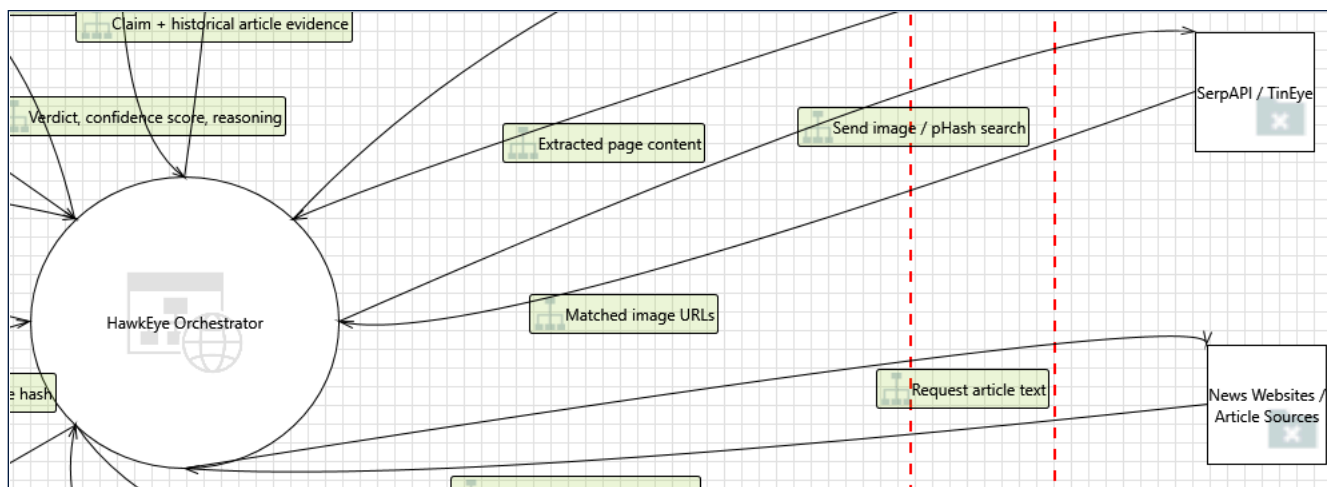
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

Interaction: Matched image URLs



49. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#ciper-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector

ciphers"><https://aka.ms/tmtcrypto#vector-ciphers> Use approved asymmetric algorithms, key lengths, and padding. Refer: <https://aka.ms/tmtcrypto#padding> Use approved random number generators. Refer: <https://aka.ms/tmtcrypto#numgen> Do not use symmetric stream ciphers. Refer: <https://aka.ms/tmtcrypto#stream-ciphers> Use approved MAC/HMAC/keyed hash algorithms. Refer: <https://aka.ms/tmtcrypto#mac-hash> Use only approved cryptographic hash functions. Refer: <https://aka.ms/tmtcrypto#hash-functions> Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

50. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: <https://aka.ms/tmtauditlog#log-sensitive-data> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access>

SDL Phase: Implementation

51. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: <https://aka.ms/tmtxmgmt#messages> Implement Default error handling page. Refer: <https://aka.ms/tmtxmgmt#default> Set Deployment Method to Retail in IIS. Refer: <https://aka.ms/tmtxmgmt#deployment> Exceptions should fail safely. Refer: <https://aka.ms/tmtxmgmt#fail> ASP.NET applications must disable tracing and debugging prior to deployment. Refer: <https://aka.ms/tmtconfigmgmt#trace-deploy> Implement controls to prevent username enumeration. Refer: <https://aka.ms/tmtauthn#controls-username-enum>

SDL Phase: Implementation

52. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: <https://aka.ms/tmtauditlog#auditing> Ensure that log rotation and separation are in place. Refer: <https://aka.ms/tmtauditlog#log-rotation> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access> Ensure that User Management Events are Logged. Refer: <https://aka.ms/tmtauditlog#user-management>

SDL Phase: Implementation

53. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

54. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials

are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: <a href="https://aka.ms/tmtdat#autocomplete-

Mitigation(s): input">https://aka.ms/tmtdat#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

55. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <a href="https://aka.ms/tmtcommsec#x509-

Mitigation(s): ssltls">https://aka.ms/tmtcommsec#x509-ssltls Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

56. An adversary may spoof SerpAPI / TinEye and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Consider using a standard authentication mechanism to authenticate to Web Application. Refer: <a href="https://aka.ms/tmtauthn#standard-

Mitigation(s): authn-web-app">https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

57. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

Mitigation(s): href="https://aka.ms/tmtinputval#typesafe">https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

58. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

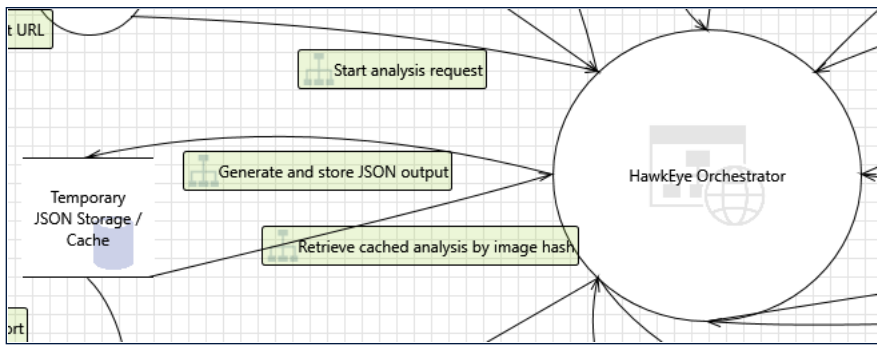
Justification: <no mitigation provided>

Possible Encrypt sections of Web App's configuration files that contain sensitive data. Refer: <a href="https://aka.ms/tmtdat#encrypt-

Mitigation(s): data">https://aka.ms/tmtdat#encrypt-data

SDL Phase: Implementation

Interaction: Retrieve cached analysis by image hash



59. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.
Justification: <no mitigation provided>
Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data
SDL Phase: Implementation

60. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering
Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe
SDL Phase: Implementation

61. An adversary may spoof Temporary JSON Storage / Cache and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing
Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application
Justification: <no mitigation provided>
Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app
SDL Phase: Design

62. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing
Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe
SDL Phase: Implementation

63. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing
Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,
Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer:

SDL Phase: Implementation

64. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing
Description: Ensure that TLS certificate parameters are configured with correct values
Justification: <no mitigation provided>
Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer:
SDL Phase: Implementation

65. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation
Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer:
SDL Phase: Implementation

66. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details
Justification: <no mitigation provided>
Possible Mitigation(s): Do not expose security details in error messages. Refer:
SDL Phase: Implementation

67. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure
Description: An adversary may gain access to sensitive data from log files
Justification: <no mitigation provided>
Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer:
SDL Phase: Implementation

68. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

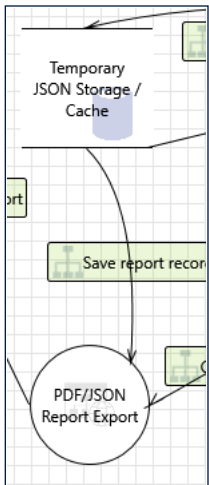
Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

Interaction: Save report record



69. An adversary may spoof Temporary JSON Storage / Cache and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

Mitigation(s): https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

70. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

Mitigation(s): Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

71. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdat#autocomplete-input

Mitigation(s): Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

72. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

Mitigation(s): https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

73. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing

Mitigation(s): Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

74. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

75. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data

Mitigation(s): Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

76. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

77. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

Mitigation(s): href="https://aka.ms/tmtinputval#typesafe">https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

78. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

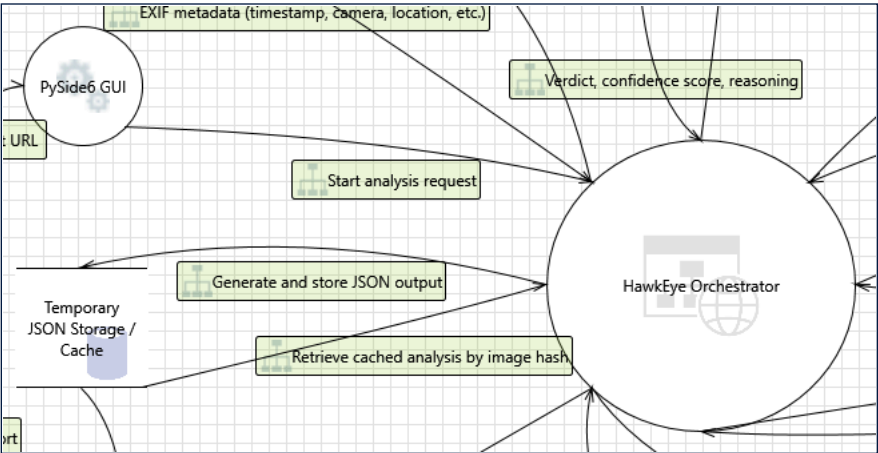
Justification: <no mitigation provided>

Possible Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

Mitigation(s): data">https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation

Interaction: Start analysis request



79. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: <https://aka.ms/tmtxmgmt#messages> Implement Default error handling page. Refer: <https://aka.ms/tmtxmgmt#default> Set Deployment Method to Retail in IIS. Refer: <https://aka.ms/tmtxmgmt#deployment> Use only approved symmetric block ciphers and key lengths. Refer: <https://aka.ms/tmtcrypto#cipher-length> Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: <https://aka.ms/tmtcrypto#vector-ciphers> Use approved asymmetric algorithms, key lengths, and padding. Refer: <https://aka.ms/tmtcrypto#padding> Use approved random number generators. Refer: <https://aka.ms/tmtcrypto#numgen> Do not use symmetric stream ciphers. Refer: <https://aka.ms/tmtcrypto#stream-ciphers> Use approved MAC/HMAC/keyed hash algorithms. Refer: <https://aka.ms/tmtcrypto#mac-hash> Use only approved cryptographic hash functions. Refer: <https://aka.ms/tmtcrypto#hash-functions> Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

80. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that the application does not log sensitive user data. Refer: <https://aka.ms/tmtauditlog#log-sensitive-data> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access>

SDL Phase: Implementation

81. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Mitigation(s): Do not expose security details in error messages. Refer: <https://aka.ms/tmtxmgmt#messages> Implement Default error handling page. Refer: <https://aka.ms/tmtxmgmt#default> Set Deployment Method to Retail in IIS. Refer: <https://aka.ms/tmtxmgmt#deployment> Exceptions should fail safely. Refer: <https://aka.ms/tmtxmgmt#fail> ASP.NET applications must disable tracing and debugging prior to deployment. Refer: <https://aka.ms/tmtconfigmgmt#trace-deploy> Implement controls to prevent username enumeration. Refer: <https://aka.ms/tmtauthn#controls-username-enum>

SDL Phase: Implementation

82. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: <https://aka.ms/tmtauditlog#auditing> Ensure that log rotation and separation are in place. Refer: <https://aka.ms/tmtauditlog#log-rotation> Ensure that Audit and Log Files have Restricted Access. Refer: <https://aka.ms/tmtauditlog#log-restricted-access> Ensure that User Management Events are Logged. Refer: <https://aka.ms/tmtauditlog#user-management>

SDL Phase: Implementation

83. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: <https://aka.ms/tmtcommsec#x509-sslts>

SDL Phase: Implementation

84. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdat#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

85. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences. Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

86. An adversary may spoof PySide6 GUI and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

87. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

88. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

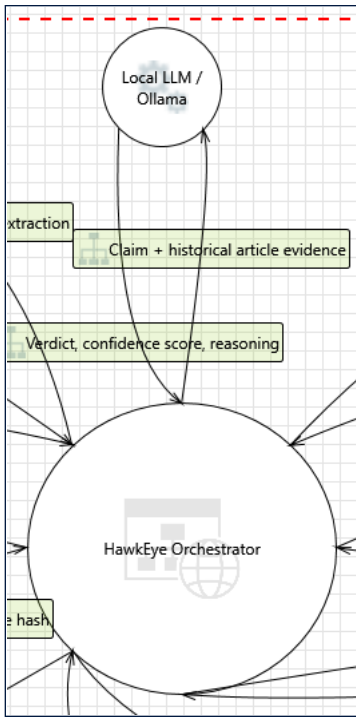
Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdat#encrypt-data

SDL Phase: Implementation

Interaction: Verdict, confidence score, reasoning



89. An adversary can reverse weakly encrypted or hashed content [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can reverse weakly encrypted or hashed content

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Use only approved symmetric block ciphers and key lengths. Refer: https://aka.ms/tmtcrypto#cipher-length Use approved block cipher modes and initialization vectors for symmetric ciphers. Refer: https://aka.ms/tmtcrypto#vector-ciphers Use approved asymmetric algorithms, key lengths, and padding. Refer: https://aka.ms/tmtcrypto#padding Use approved random number generators. Refer: https://aka.ms/tmtcrypto#numgen Do not use symmetric stream ciphers. Refer: https://aka.ms/tmtcrypto#stream-ciphers Use approved MAC/HMAC/keyed hash algorithms. Refer: https://aka.ms/tmtcrypto#mac-hash Use only approved cryptographic hash functions. Refer: https://aka.ms/tmtcrypto#hash-functions Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

90. An adversary may gain access to sensitive data from log files [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary may gain access to sensitive data from log files

Justification: <no mitigation provided>

Possible Ensure that the application does not log sensitive user data. Refer: https://aka.ms/tmtauditlog#log-sensitive-data

Mitigation(s): Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access

SDL Phase: Implementation

91. An adversary can gain access to sensitive information through error messages [State: Not Started] [Priority: High]

Category: Information Disclosure

Description: An adversary can gain access to sensitive data such as the following, through verbose error messages - Server names - Connection strings - Usernames - Passwords - SQL procedures - Details of dynamic SQL failures - Stack trace and lines of code - Variables stored in memory - Drive and folder locations - Application install points - Host configuration settings - Other internal application details

Justification: <no mitigation provided>

Possible Do not expose security details in error messages. Refer: https://aka.ms/tmtxmgmt#messages

Mitigation(s): Implement Default error handling page. Refer: https://aka.ms/tmtxmgmt#default Set Deployment Method to Retail in IIS. Refer: https://aka.ms/tmtxmgmt#deployment Exceptions should fail

safely. Refer: https://aka.ms/tmtxmgmt#fail ASP.NET applications must disable tracing and debugging prior to deployment. Refer: https://aka.ms/tmtconfigmgmt#trace-deploy Implement controls to prevent username enumeration. Refer: https://aka.ms/tmtauthn#controls-username-enum

SDL Phase: Implementation

92. Attacker can deny the malicious act and remove the attack foot prints leading to repudiation issues [State: Not Started] [Priority: Medium]

Category: Repudiation

Description: Proper logging of all security events and user actions builds traceability in a system and denies any possible repudiation issues. In the absence of proper auditing and logging controls, it would become impossible to implement any accountability in a system

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that auditing and logging is enforced on the application. Refer: https://aka.ms/tmtauditlog#auditing Ensure that log rotation and separation are in place. Refer: https://aka.ms/tmtauditlog#log-rotation Ensure that Audit and Log Files have Restricted Access. Refer: https://aka.ms/tmtauditlog#log-restricted-access Ensure that User Management Events are Logged. Refer: https://aka.ms/tmtauditlog#user-management

SDL Phase: Implementation

93. An adversary can spoof the target web application due to insecure TLS certificate configuration [State: Not Started] [Priority: High]

Category: Spoofing

Description: Ensure that TLS certificate parameters are configured with correct values

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts

SDL Phase: Implementation

94. An adversary can steal sensitive data like user credentials [State: Not Started] [Priority: High]

Category: Spoofing

Description: Attackers can exploit weaknesses in system to steal user credentials. Downstream and upstream components are often accessed by using credentials stored in configuration stores. Attackers may steal the upstream or downstream component credentials. Attackers may steal credentials if, Credentials are stored and sent in clear text, Weak input validation coupled with dynamic sql queries, Password retrieval mechanism are poor,

Justification: <no mitigation provided>

Possible Mitigation(s): Explicitly disable the autocomplete HTML attribute in sensitive forms and inputs. Refer: https://aka.ms/tmtdata#autocomplete-input Perform input validation and filtering on all string type Model properties. Refer: https://aka.ms/tmtinputval#typemodel Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe Enable step up or adaptive authentication. Refer: https://aka.ms/tmtauthn#step-up-adaptive-authn Implement forgot password functionalities securely. Refer: https://aka.ms/tmtauthn#forgot-pword-fxn Ensure that password and account policy are implemented. Refer: https://aka.ms/tmtauthn#pword-account-policy Implement input validation on all string type parameters accepted by Controller methods. Refer: https://aka.ms/tmtinputval#string-method

SDL Phase: Implementation

95. An adversary can create a fake website and launch phishing attacks [State: Not Started] [Priority: High]

Category: Spoofing

Description: Phishing is attempted to obtain sensitive information such as usernames, passwords, and credit card details (and sometimes, indirectly, money), often for malicious reasons, by masquerading as a Web Server which is a trustworthy entity in electronic communication

Justification: <no mitigation provided>

Possible Mitigation(s): Verify X.509 certificates used to authenticate SSL, TLS, and DTLS connections. Refer: https://aka.ms/tmtcommsec#x509-sslts Ensure that authenticated ASP.NET pages incorporate UI Redressing or clickjacking defences.

Refer: https://aka.ms/tmtconfigmgmt#ui-defenses Validate all redirects within the application are closed or done safely. Refer: https://aka.ms/tmtinputval#redirect-safe

SDL Phase: Implementation

96. An adversary may spoof Local LLM / Ollama and gain access to Web Application [State: Not Started] [Priority: High]

Category: Spoofing

Description: If proper authentication is not in place, an adversary can spoof a source process or external entity and gain unauthorized access to the Web Application

Justification: <no mitigation provided>

Possible Mitigation(s): Consider using a standard authentication mechanism to authenticate to Web Application. Refer: https://aka.ms/tmtauthn#standard-authn-web-app

SDL Phase: Design

97. An adversary can gain access to sensitive data by performing SQL injection through Web App [State: Not Started] [Priority: High]

Category: Tampering

Description: SQL injection is an attack in which malicious code is inserted into strings that are later passed to an instance of SQL Server for parsing and execution. The primary form of SQL injection consists of direct insertion of code into user-input variables that are concatenated with SQL commands and executed. A less direct attack injects malicious code into strings that are destined for storage in a table or as metadata. When the stored strings are subsequently concatenated into a dynamic SQL command, the malicious code is executed.

Justification: <no mitigation provided>

Possible Mitigation(s): Ensure that type-safe parameters are used in Web Application for data access. Refer: https://aka.ms/tmtinputval#typesafe

SDL Phase: Implementation

98. An adversary can gain access to sensitive data stored in Web App's config files [State: Not Started] [Priority: High]

Category: Tampering

Description: An adversary can gain access to the config files. and if sensitive data is stored in it, it would be compromised.

Justification: <no mitigation provided>

Possible Mitigation(s): Encrypt sections of Web App's configuration files that contain sensitive data. Refer: https://aka.ms/tmtdata#encrypt-data

SDL Phase: Implementation